

Road Network Navigation in Any Pose

Road Network Navigation in Any Pose

1. Course Content
2. Preparation
3. Road Network Planning and Navigation in Any Pose
 - 3.1 Case Demonstration
 - 3.2 Requesting Navigation Through the Action Service Interface
4. Source Code Analysis
5. Parameter Debugging

1. Course Content

Basic

- Start the road network navigation feature, learn to use the roadnet_nav visualization panel to send target poses, and combine it with the road network planning feature to reach any target pose and orientation.

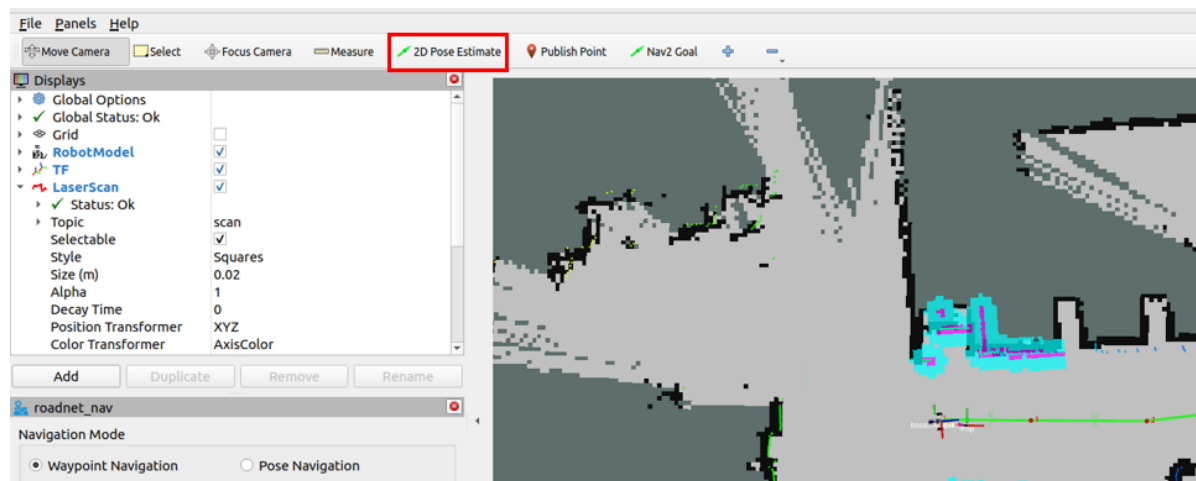
Advanced

- Learn how to request navigation through the action service interface
- Understand the source code implementation logic

2. Preparation

```
ros2 launch microros_v2_bringup roadnet_nav.launch.py
```

- Click **2D Pose Estimate** to initialize the vehicle's pose on the map



3. Road Network Planning and Navigation in Any Pose

- Here we use road network navigation by publishing the `/road_net_nav` topic on the terminal. This topic accepts a `PoseStamped` target pose, travels along the road network to a point near the target pose, and then accurately adjusts to the target pose. The topic publishing format is as follows:

```
ros2 topic pub -1 /road_net_nav geometry_msgs/PoseStamped "{
  pose: {
    position: {x: xx, y: xx, z: 0.0},
    orientation: {x: 0.0, y: 0.0, z: xx, w: xx}
  }
}"
```

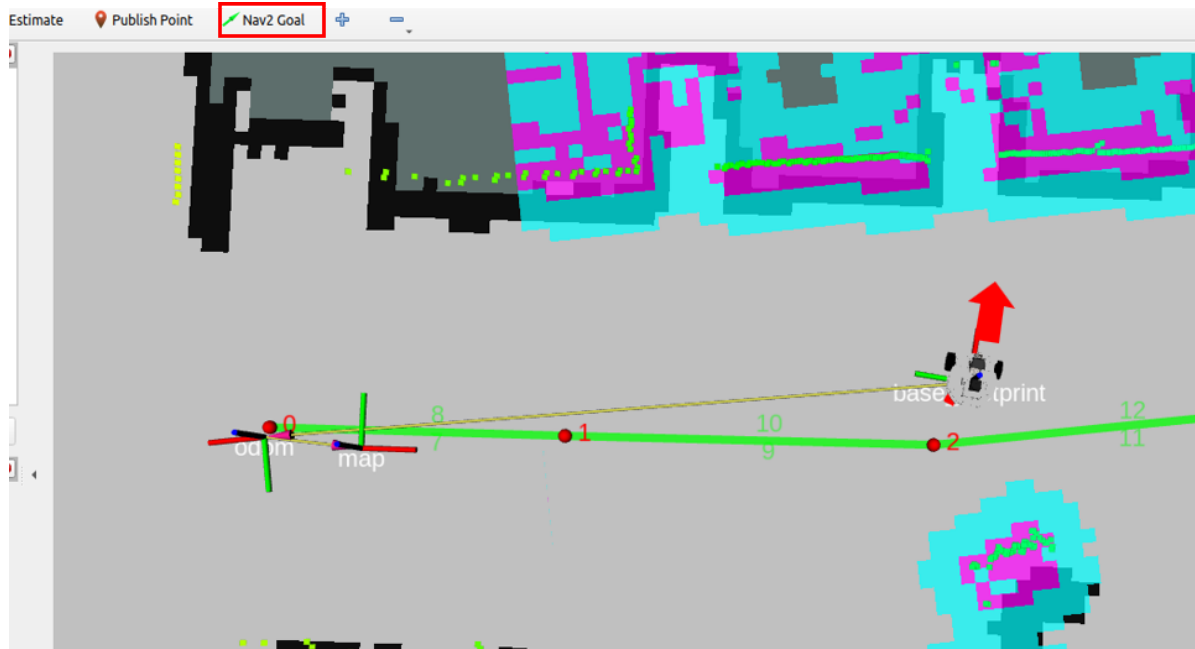
[!NOTE]

A pose consists of position and orientation. For a ground mobile robot, the position only has the planar x, y coordinates, and the orientation is only the rotation about its own z-axis. In the pose:

- **position: {x: xx, y: xx, z: 0.0}** is the position information
- **orientation: {x: 0.0, y: 0.0, z: xx, w: xx}** is the vehicle rotation information described by a quaternion

3.1 Case Demonstration

- Use the **Nav2 Goal** tool to navigate to a pose point first



- Use the following command to get the current coordinates of the robot in the map

```
ros2 run tf2_ros tf2_echo map base_footprint
```

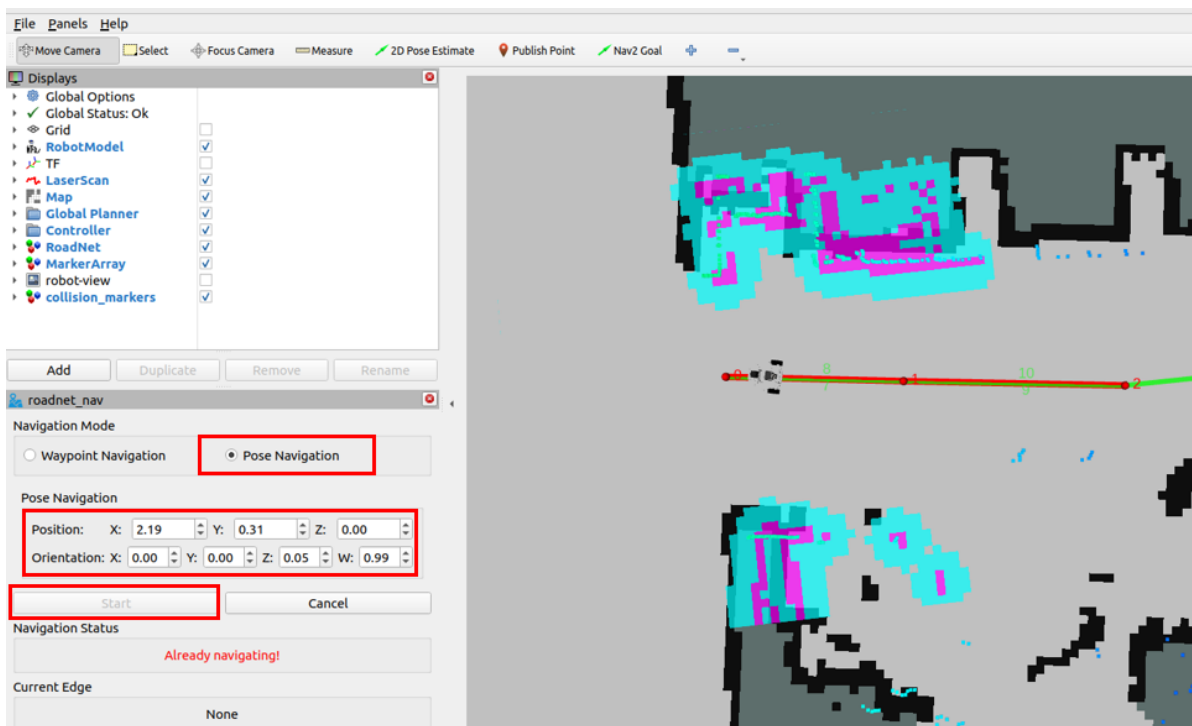
Where **Translation** and **Rotation: in Quaternion (xyzw)** are the pose of the robot vehicle in the global map.

```

ahboom: ~
jetson@yahboom: ~ 70x19
- Rotation: in RPY (radian) [0.000, 0.000, -0.104]
- Rotation: in RPY (degree) [0.000, 0.000, -5.946]
- Matrix:
  0.995  0.104  0.000  2.383
 -0.104  0.995 -0.000 -1.352
 -0.000  0.000  1.000  0.000
  0.000  0.000  0.000  1.000
At time 1763608206.239000000
- Translation: [2.383, -1.352, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.052, 0.999]
- Rotation: in RPY (radian) [0.000, 0.000, -0.104]
- Rotation: in RPY (degree) [0.000, 0.000, -5.959]
- Matrix:
  0.995  0.104  0.000  2.383
 -0.104  0.995 -0.000 -1.352
 -0.000  0.000  1.000  0.000
  0.000  0.000  0.000  1.000
^C[INFO] [1763608207.095851031] [rclcpp]: signal_handler(signum=2)
jetson@yahboom:~$

```

- Then we let the robot return to the initial position, and in the **roadnet_nav** panel select **Pose Navigation**, fill in the pose queried just now, and click **Start** to begin navigation



- Afterwards, the robot vehicle will first travel along the road network to a waypoint near the target point, then navigate along the road network edges, and finally adjust to the target pose.

3.2 Requesting Navigation Through the Action Service Interface

- Replace the pose parameter with the pose queried earlier to achieve the same functionality as the **roadnet_nav** panel

```
ros2 action send_goal --feedback /compute_route_and_follow
roadnet/action/RoadNetToGoal \
"{
  use_poses: true,
  goal: {
    header: {frame_id: 'map'},
    pose: {position: {x: 2.19, y: 0.31, z: 0.0}, orientation: {z: 0.05, w:
0.99}}
  }
}"
```

4. Source Code Analysis

- Refer to the source code analysis in the Waypoint-to-Waypoint Planning Navigation chapter; the `computeRouteAndFollow` function functionality includes the pose navigation part

5. Parameter Debugging

- Refer to the parameter debugging in the Waypoint-to-Waypoint Planning Navigation chapter; the parameter file is the same